



17.6. Graph Problems

17.6.1. Shortest Path in Un-Weighted Graph

Problem :

- ✓ Given UnDirected Un-Weighted Graph and Source Vertex.
- ✓ Print the Shortest Distances from given Source Vertex to every other Vertex.

UndirectedGraph.java

```
package com.myjlc.graphs.problems;

import java.util.ArrayList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class UndirectedGraph {
    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    // Create a graph
    public UndirectedGraph(int vertexCount) {
        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }

    // Add edges to the graph
    public void addEdge(int vertex1, int vertex2) {
        adjList.get(vertex1).add(vertex2);
        adjList.get(vertex2).add(vertex1);
    }
}
```



//Print the Graph

```
public void printGraph() {  
  
    for (int i = 0; i < adjList.size(); i++) {  
        System.out.print(i+ "\t => ");  
        for (int j = 0; j < adjList.get(i).size(); j++) {  
            System.out.print(adjList.get(i).get(j) + "\t");  
        }  
        System.out.println("");  
    }  
}
```

Lab9.java

```
package com.myjlc.graphs.problems;
```

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
**/
```

```
public class Lab9 {
```

```
public static void printShortestPaths(UndirectedGraph mygraph,int  
sourceVertex) {
```

```
    int distance[] = new int[mygraph.vertexCount] ;
```

```
    for(int i=0;i<distance.length;i++) {  
        distance[i] = Integer.MAX_VALUE;  
    }
```

```
    Queue<Integer> myque = new LinkedList<Integer>();  
    boolean visited[] = new boolean[mygraph.vertexCount];
```



```
distance[sourceVertex] = 0;
visited[sourceVertex] = true;
myqueue.add(sourceVertex);

while (!myqueue.isEmpty()) {
    int currVertex = myqueue.poll();

    for (int vertex : mygraph.adjList.get(currVertex)) {
        if (!visited[vertex]) {
            distance[vertex] = distance[currVertex] + 1;
            visited[vertex] = true;
            myqueue.add(vertex);
        }
    }
}
for(int i=0;i<distance.length;i++) {
    System.out.println(distance[i]);
}

}

public static void main(String[] args) {

    UndirectedGraph mygraph = new UndirectedGraph(5);
    mygraph.addEdge(0, 1);
    mygraph.addEdge(0, 2);
    mygraph.addEdge(1, 2);
    mygraph.addEdge(1, 3);
    mygraph.addEdge(2, 3);
    mygraph.addEdge(2, 4);
    mygraph.addEdge(3,4);

    printShortestPaths(mygraph,0);
}
}
```



17.6.2. Topological Ordering of DAG

Problem :

- ✓ Given **DAG(Directed Acyclic Graph)**.
- ✓ You need to find the Topological Ordering/ Sorting of given **DAG**.
- ✓ **BFS based Solution (Kahns Algorithm)**

DirectedGraph.java

```
package com.myjlc.graphs.problems;

import java.util.ArrayList;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class DirectedGraph {
    int vertexCount;
    ArrayList<ArrayList<Integer>> adjList;

    // Create a graph
    public DirectedGraph(int vertexCount) {

        this.vertexCount = vertexCount;
        this.adjList = new ArrayList<ArrayList<Integer>>(vertexCount);

        for (int i = 0; i < vertexCount; i++) {
            this.adjList.add(new ArrayList<Integer>());
        }
    }

    // Add edges to the graph
    public void addEdge(int vertex1, int vertex2) {
        adjList.get(vertex1).add(vertex2);
    }
}
```



//Print the Graph

```
public void printGraph() {  
  
    for (int i = 0; i < adjList.size(); i++) {  
        System.out.print(i+ "\t => ");  
        for (int j = 0; j < adjList.get(i).size(); j++) {  
            System.out.print(adjList.get(i).get(j) + "\t");  
        }  
        System.out.println("");  
    }  
}
```

Lab10.java

```
package com.myjlc.graphs.problems;
```

```
import java.util.LinkedList;
```

```
import java.util.Queue;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @Company: Java Learning Center
```

```
*/
```

```
public class Lab10 {
```

```
    public static void topologicalOrder(DirectedGraph mygraph) {
```

```
        int indegrees[] = new int[mygraph.vertexCount] ;
```

```
        for(int i=0;i<indegrees.length;i++) {  
            for(int vertex:mygraph.adjList.get(i))  
                indegrees[vertex] ++;  
        }
```

```
        Queue<Integer> myque = new LinkedList<Integer>();
```



```
        for(int i=0;i<indegrees.length;i++) {
            if(indegrees[i]==0)
                myque.add(i);
        }

        while (!myque.isEmpty()) {
            int currVertex = myque.poll();
            System.out.println(currVertex);

            for (int vertex : mygraph.adjList.get(currVertex)) {
                indegrees[vertex]--;
                if(indegrees[vertex]==0)
                    myque.add(vertex);
            }
        }
    }

    public static void main(String[] args) {

        DirectedGraph mygrah = new DirectedGraph(5);

        mygrah.addEdge(0, 2);
        mygrah.addEdge(0, 3);

        mygrah.addEdge(1, 3);
        mygrah.addEdge(1, 4);

        mygrah.addEdge(2, 3);

        topologicalOrder(mygrah);
    }
}
```



17.6.3. Topological Ordering of DAG

Problem :

- ✓ Given **DAG(Directed Acyclic Graph)**.
- ✓ You need to find the Topological Ordering/ Sorting of given **DAG**.
- ✓ **DFS** based Solution

Lab11.java

```
package com.myjlc.graphs.problems;

import java.util.Stack;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab11 {

    public static void topologicalOrder(DirectedGraph mygraph) {

        boolean visited[] = new boolean[mygraph.vertexCount];
        Stack<Integer> mystack = new Stack<Integer>();

        for (int i = 0; i < mygraph.vertexCount; i++) {
            if (!visited[i])
                dfs(mygraph, i, visited, mystack);
        }

        while(!mystack.isEmpty()) {
            int vertex = mystack.pop();
            System.out.println(vertex);
        }
    }
}
```



```
public static void dfs( DirectedGraph mygraph, int currVertex,  
boolean visited[], Stack<Integer> mystack ) {
```

```
    visited[currVertex] = true;
```

```
    for (int vertex : mygraph.adjList.get(currVertex)) {  
        if (!visited[vertex])  
            dfs(mygraph, vertex, visited, mystack);  
    }
```

```
    mystack.push(currVertex);  
}
```

```
public static void main(String[] args) {
```

```
    DirectedGraph mygrah = new DirectedGraph(5);
```

```
    mygrah.addEdge(0, 2);  
    mygrah.addEdge(0, 3);
```

```
    mygrah.addEdge(1, 3);  
    mygrah.addEdge(1, 4);
```

```
    mygrah.addEdge(2, 3);
```

```
    topologicalOrder(mygrah);  
}
```

```
}
```

17.6.4. Detect Cycle in Undirected Graph

Problem :

- ✓ Given **Undirected Graph**.
- ✓ You need to Check whether Cycle is Present or not.
- ✓ **DFS** based Solution

Lab12.java

```
package com.myjlc.graphs.problems;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
public class Lab12 {

    public static boolean isCyclePresent(UndirectedGraph mygraph) {

        boolean visited[] = new boolean[mygraph.vertexCount];

        for (int i = 0; i < mygraph.vertexCount; i++) {
            if (!visited[i]) {
                if(dfs(mygraph, i, visited, -1))
                    return true;
            }
        }
        return false;
    }
}
```



```
public static boolean dfs(UndirectedGraph mygraph, int
currVertex, boolean[] visited, int parent) {

    visited[currVertex] = true;

    for (int vertex : mygraph.adjList.get(currVertex)) {
        if (!visited[vertex]) {
            if (dfs(mygraph, vertex, visited, currVertex))
                return true;
        } else {
            if (vertex != parent)
                return true;
        }
    }
    return false;
}

public static void main(String[] args) {

    UndirectedGraph mygrah = new UndirectedGraph(5);

    mygrah.addEdge(0, 1);

    mygrah.addEdge(1, 2);
    mygrah.addEdge(1, 3);

    mygrah.addEdge(2, 3);
    mygrah.addEdge(2, 4);

    boolean flag = isCyclePresent(mygrah);
    System.out.println(flag);
}
```